SafeRide: Project Report

SafeRide is a full-stack ride-booking platform (customer, driver and admin apps) built around one trust guarantee: once a customer confirms a fare, the driver cannot charge more. The project is at Phase 16 of 18, with everything up to CI/CD and Docker packaging done and cloud deployment next.

Project at a glance

Item	Detail
Purpose	Ride booking with a locked, tamper-proof fare and built-in safety and dispute tools
Users	Customers, Drivers, Admins (role-based access)
Architecture	Modular monolith: one Spring Boot backend plus a React single-page app
Backend size	About 9,000 lines of Java across 228 source files, in 20 business modules
Frontend size	About 9,700 lines of TypeScript/React, 23 pages
Automated tests	134 backend tests reported passing in the project README, plus frontend component tests
Delivery	Docker Compose full stack, GitHub Actions CI/CD publishing images to GitHub Container Registry
Progress	16 of 18 planned phases complete

Tools and technology

Layer	Technology	Used for
Backend language/framework	Java 21, Spring Boot 3.5, Maven	Core application and build

Layer	Technology	Used for
Backend libraries	Spring Web, Spring Data JPA/Hibernate, Spring Security, Spring WebSocket (STOMP), Spring Kafka, Spring Actuator, Bean Validation, Lombok	REST APIs, database access, security, live updates, events, health checks
Authentication	JWT (jjwt) with rotating refresh tokens	Login and session security
API documentation	springdoc-openapi (Swagger UI)	Interactive API docs
Database	MySQL 8	Source of truth for all durable data
Cache / geo	Redis 7	Driver geo-location index for matching, login rate limiting
Messaging	Apache Kafka (optional, additive)	Mirrors domain events for external consumers; the app runs without it
Frontend	React 18, TypeScript, Vite	Web app for all three roles
UI	Tailwind CSS, shadcn/ui (Radix), Lucide icons, Framer Motion	Styling, components, animation
Frontend data	TanStack Query, Axios, React Router, React Hook Form + Zod	API calls, routing, validated forms
Maps	Leaflet, React-Leaflet, OpenStreetMap	Live map, markers, route lines
Charts	Recharts	Admin analytics
Real-time	@stomp/stompjs over WebSocket	Live ride status and driver location
Backend testing	JUnit 5, Mockito, Testcontainers (real MySQL and Redis), embedded Kafka	Unit and integration tests
Frontend testing	Vitest, React Testing Library, ESLint, strict TypeScript	Component tests and code quality
DevOps	Docker, Docker Compose, GitHub Actions, GitHub Container Registry	Packaging, CI on every push, image publishing on main

Map and payment providers are built behind interfaces and currently use mock implementations. A real provider (for example Google Maps or Razorpay) can be added as a second implementation without a rewrite.

Key features: the trust layer

Feature	What it does
Fare Lock	The fare is calculated only on the backend and frozen when the customer confirms. Drivers have no way to change it. Changing the destination recalculates and unlocks the fare for re-confirmation.
Smart driver matching	Scores drivers on distance, rating, reliability, acceptance rate, ETA and cancellation history, not just nearest. Weights are configurable.
Driver Reliability Score	A 0 to 100 score updated after every ride event (acceptance, cancellation, completion, on-time, complaints).
Automatic reassignment	If a driver declines or cancels, the ride is re-matched to the next-best driver and the previous one is excluded.
Ride OTP	A 4-digit OTP, visible only to the customer, is required to start the ride.
Complaints and fare disputes	Customers and drivers can file complaints. Admins investigate, resolve or dismiss them with an audit trail.
SOS and trip sharing	One-tap SOS alert with live location that admins see. A 24-hour public tracking link for family and friends that hides the OTP and phone numbers.
Payment and rating	Payment charges exactly the locked fare. Ratings (1 to 5) feed the driver's score.

Features by role

Customer

- Register, log in (email or phone), and edit profile with photo
- Pick pickup and destination on a live map, with distance and ETA preview
- Compare vehicle types with a per-vehicle fare preview, then see the full fare breakdown
- Confirm and lock the fare, then watch an animated driver search
- Track the ride live on the map with driver details and OTP
- Cancel from the ride screen or from My Rides, pay, rate, report an issue, share the trip, send SOS
- Ride history, complaint history, Safety Center, notifications bell
- Settings including day/night mode

Driver

- Self-register with licence, add and activate vehicles, upload documents for review
- Go online or offline (only after admin verification) and report live location
- Receive assigned rides, accept or decline, arrive, start with OTP, complete
- Earnings page, ride history, editable profile, reliability and rating stats

Admin

- Dashboard with live platform numbers
- Driver verification console: approve, reject or block drivers, vehicles and documents
- User management, including creating other admin accounts
- Complaint management and SOS safety alerts (resolve or delete)
- Analytics: 15 real database-backed metrics with charts (rides, revenue, drivers online, complaints)
- Admin settings

Platform capabilities

- **Real-time updates:** ride status, driver assignments and live driver location are pushed over WebSocket, with access checks so users only see their own rides.
- **Notifications:** in-app notifications for every ride and complaint event, with unread count and mark-as-read.

- **Security:** JWT with refresh-token rotation, role-based access (Customer, Driver, Admin), login rate limiting (Redis), blocked-account checks, and a production safety check that refuses to start with a default secret.
- **Audit trail:** admin actions are logged.
- **Reliability by design:** optimistic locking prevents two drivers claiming the same ride. Redis and Kafka are optional, and the app falls back or fails open if either is unavailable.
- **Performance work:** recent commits fixed N+1 query problems on ride-list and admin endpoints and cached the per-request block check.

Quality and delivery

- **Testing:** 28 backend test files cover the fare maths, ride state machine, matching score, reliability score, authentication, WebSocket, payments, complaints, notifications and analytics. Integration tests run against real MySQL and Redis containers. The README reports 134 of 134 backend tests passing; I did not re-run them for this report.
- **CI/CD:** GitHub Actions runs backend tests, frontend lint, tests and build on every push. Pushes to main publish backend and frontend Docker images to GitHub Container Registry.
- **Run anywhere:** `docker compose up --build` starts MySQL, Redis, backend and frontend together. Demo accounts are seeded in development mode.

Roadmap and status

Phase	Scope	Status
1 to 6	Setup, auth, vehicles and driver verification, ride booking, Fare Lock, matching and reliability	Done
7 to 9	Customer, driver and admin web apps	Done
10 to 11	WebSocket real-time updates, Redis geo-matching and rate limiting	Done
12 to 13	Payment, rating, complaints, safety, Kafka events, notifications, analytics	Done
14 to 16	Testing, Docker, CI/CD	Done
17	Deployment (free-tier hosting first, then AWS)	Next
18	Production optimization	Planned

Known gaps and next steps

- Payments and maps use mock providers; real gateway and map integrations are still to do.
- Driver documents are stored as URLs; there is no file upload endpoint yet.
- The GitHub Actions workflows have not yet run on GitHub's own runners. They passed local linting and every command was run locally.
- No browser end-to-end tests yet.
- Cloud deployment (planned on AWS: ALB, RDS MySQL, ElastiCache Redis, S3, CloudWatch) is Phase 17.